SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA1304

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:100**
Duration : 1 Hour

Experiments

Code of Conduct:

IM.Vinay / 192311146 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.Vinay

SIMATS ENGINEERING
ASSESSMENT TOOLS

Sl. No.	Lab Experiments - Questions
1	Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG) S → 0A1 A → 0A | 1A | ε

ANSWERS

1. CFG: S to 0A1, quad A to 0A mid 1A mid epsilon

Language Definition: Generates all binary strings of length >= 2 that start with 0 and end with 1.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
bool is_member_g1(const char *str) {
    int len = strlen(str);
    if (len < 2) return false;
    if (str[0] != '0' || str[len - 1] != '1') return false;
    for (int i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') return false;
    }
    return true;
}
int main() {
    const char *test_cases[] = {"01", "001", "0101", "0111", "101", "000", "0", ""};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);
    printf("Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon\n");
    printf("--------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
```

```
    const char *str = test_cases[i];

    printf("Input: \"%s\" \t-> %s\n", str, is_member_g1(str) ? "ACCEPTED" : "REJECTED");

}
```

```
    return 0;

}
```

Output:

```
Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon
-----------------------------------------------------
Input:  "01"   ->  ACCEPTED
Input:  "001"  ->  ACCEPTED
Input: "0101"  -> ACCEPTED
Input: "0111"  -> ACCEPTED
Input:  "101"  ->  REJECTED
Input:  "000"  ->  REJECTED
Input:   "0"   ->   REJECTED
Input: ""        -> REJECTED
```

2 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$

ANSWERS

2. CFG: S to 0A1, quad A to 0A mid 1A mid epsilon

Language Definition: Generates all binary strings of length >= 2 that start with 0 and end with 1.

C Program:

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>

bool is_member_g1(const char *str) {

    int len = strlen(str);

    if (len < 2) return false;

    if (str[0] != '0' || str[len - 1] != '1') return false;

    for (int i = 0; i < len; i++) {

        if (str[i] != '0' && str[i] != '1') return false;

    }
```

```
    return true;
}
int main() {
    const char *test_cases[] = {"01", "001", "0101", "0111", "101", "000", "0", ""};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);
    printf("Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon\n");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
        const char *str = test_cases[i];
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g1(str) ? "ACCEPTED" : "REJECTED");
    }
```

```
    return 0;

}
```

Output:

```
Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon
-----------------------------------------------------
Input:  "01"   ->  ACCEPTED
Input:  "001"  ->  ACCEPTED
Input: "0101"  -> ACCEPTED
Input: "0111"  -> ACCEPTED
Input:  "101"  ->  REJECTED
Input:  "000"  ->  REJECTED
Input:   "0"   ->   REJECTED
Input: ""        -> REJECTED
```

3 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S0 \mid A \qquad A \rightarrow 1A \mid \varepsilon$

ANSWERS

3. CFG: S to 0A1, quad A to 0A mid 1A mid epsilon

Language Definition: Generates all binary strings of length >= 2 that start with 0 and end with 1.

C Program:

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>

bool is_member_g1(const char *str) {

    int len = strlen(str);

    if (len < 2) return false;

    if (str[0] != '0' || str[len - 1] != '1') return false;

    for (int i = 0; i < len; i++) {

        if (str[i] != '0' && str[i] != '1') return false;

    }
```

```
    return true;
}
int main() {
    const char *test_cases[] = {"01", "001", "0101", "0111", "101", "000", "0", ""};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);
    printf("Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon\n");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
        const char *str = test_cases[i];
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g1(str) ? "ACCEPTED" : "REJECTED");
    }
```

```
    return 0;

}
```

Output:

```
Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon
-----------------------------------------------------
Input:  "01"   ->  ACCEPTED
Input:  "001"  ->  ACCEPTED
Input: "0101"  -> ACCEPTED
Input: "0111"  -> ACCEPTED
Input:  "101"  ->  REJECTED
Input:  "000"  ->  REJECTED
Input:   "0"   ->   REJECTED
Input: ""        -> REJECTED
```

4 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S1 \mid \varepsilon$

ANSWERS

4. CFG: S to 0A1, quad A to 0A mid 1A mid epsilon

Language Definition: Generates all binary strings of length >= 2 that start with 0 and end with 1.

C Program:

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>

bool is_member_g1(const char *str) {

    int len = strlen(str);

    if (len < 2) return false;

    if (str[0] != '0' || str[len - 1] != '1') return false;

    for (int i = 0; i < len; i++) {

        if (str[i] != '0' && str[i] != '1') return false;

    }
```

```
    return true;
}
int main() {
    const char *test_cases[] = {"01", "001", "0101", "0111", "101", "000", "0", ""};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);
    printf("Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon\n");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
        const char *str = test_cases[i];
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g1(str) ? "ACCEPTED" : "REJECTED");
    }
```

```
    return 0;
}
```

Output:

```
Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon
-----------------------------------------------------
Input:  "01"   ->  ACCEPTED
Input:  "001"  ->  ACCEPTED
Input: "0101"  -> ACCEPTED
Input: "0111"  -> ACCEPTED
Input:  "101"  ->  REJECTED
Input:  "000"  ->  REJECTED
Input:   "0"   ->   REJECTED
Input: ""        -> REJECTED
```

5 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow A101A \qquad A \rightarrow 0A \mid 1A \mid \varepsilon$

ANSWERS

5. CFG: S to 0A1, quad A to 0A mid 1A mid epsilon

Language Definition: Generates all binary strings of length >= 2 that start with 0 and end with 1.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
bool is_member_g1(const char *str) {
    int len = strlen(str);
    if (len < 2) return false;
    if (str[0] != '0' || str[len - 1] != '1') return false;
    for (int i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') return false;
    }
```

```
    return true;
}
int main() {
    const char *test_cases[] = {"01", "001", "0101", "0111", "101", "000", "0", ""};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);
    printf("Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon\n");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
        const char *str = test_cases[i];
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g1(str) ? "ACCEPTED" : "REJECTED");
    }
```

```
    return 0;
}
```

Output:

```
Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon
-----------------------------------------------------
Input:  "01"   ->  ACCEPTED
Input:  "001"  ->  ACCEPTED
Input: "0101"  -> ACCEPTED
Input: "0111"  -> ACCEPTED
Input:  "101"  ->  REJECTED
Input:  "000"  ->  REJECTED
Input:   "0"   ->   REJECTED
Input: ""        -> REJECTED
```

ANSWERS

6. CFG: S to 0A1, quad A to 0A mid 1A mid epsilon

Language Definition: Generates all binary strings of length >= 2 that start with 0 and end with 1.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
bool is_member_g1(const char *str) {
    int len = strlen(str);
    if (len < 2) return false;
    if (str[0] != '0' || str[len - 1] != '1') return false;
    for (int i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') return false;
    }
    return true;
```

```
}

int main() {

    const char *test_cases[] = {"01", "001", "0101", "0111", "101", "000", "0", ""};

    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);

    printf("Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon\n");

    printf("-------------------------------------------------\n");

    for (int i = 0; i < num_tests; i++) {

        const char *str = test_cases[i];

        printf("Input: \"%s\" \t-> %s\n", str, is_member_g1(str) ? "ACCEPTED" : "REJECTED");

    }
```

```
    return 0;

}
```

Output:

```
Grammar 1: S -> 0A1, A -> 0A | 1A | epsilon
-----------------------------------------------------
Input:  "01"   ->  ACCEPTED
Input:  "001"  ->  ACCEPTED
Input: "0101"  -> ACCEPTED
Input: "0111"  -> ACCEPTED
Input:  "101"  ->  REJECTED
Input:  "000"  ->  REJECTED
Input:   "0"   ->   REJECTED
Input: ""        -> REJECTED
```

7. CFG: S to 0S0 mid 1S1 mid 0 mid 1 mid epsilon

Language Definition: Generates all binary palindromes (including odd-length, even-length, and empty strings).

C Program:

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>

bool is_member_g2(const char *str) {

    int len = strlen(str);

      for (int i = 0; i < len; i++) {

        if (str[i] != '0' && str[i] != '1') return false;

    }

    int start = 0, end = len - 1;

    while (start < end) {

        if (str[start] != str[end]) return false;

        start++;
```

```
        end--;
    }
    return true;
}

int main() {
    const char *test_cases[] = {"", "0", "1", "00", "11", "010", "1001", "011010", "1010"};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);

    printf("Grammar 2: S -> 0S0 | 1S1 | 0 | 1 | epsilon\n");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
        const char *str = test_cases[i];
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g2(str) ? "ACCEPTED" : "REJECTED");
    }
    return 0;
}
```

Output:

```
Grammar 2: S -> 0S0 | 1S1 | 0 | 1 | epsilon
-------------------------------------------------
Input:    ""     ->    ACCEPTED
Input:   "0"     ->    ACCEPTED
Input:   "1"     ->    ACCEPTED
Input:  "00"     ->    ACCEPTED
Input:  "11"     ->    ACCEPTED
Input:  "010"    ->    ACCEPTED
Input: "1001"    ->    ACCEPTED
```

Input: "011010" -> REJECTED

Input: "1010" -> REJECTED

8. CFG: S to 0S0 mid A, quad A to 1A mid epsilon

Language Definition: Generates strings of the form 0^n 1^m 0^n where n >= 0, m >= 0.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
bool is_member_g3(const char *str) {
    int len = strlen(str);
    int i = 0, j = len - 1;
    for (int k = 0; k < len; k++) {
        if (str[k] != '0' && str[k] != '1') return false;
    }
    while (i < j && str[i] == '0' && str[j] == '0') {
        i++;
        j--;
    }
    for (int k = i; k <= j; k++) {
        if (str[k] != '1') return false;
    }
    return true;
}
```

```
int main() {
    const char *test_cases[] = {"", "111", "00", "010", "0011100", "000", "01100", "0100"};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);
    printf("Grammar 3: S -> 0S0 | A, A -> 1A | epsilon\n");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
        const char *str = test_cases[i];
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g3(str) ? "ACCEPTED" : "REJECTED");
    }
    return 0;
}
```

Output:

```
Grammar 3: S -> 0S0 | A, A -> 1A | epsilon
-------------------------------------------------
Input: ""          -> ACCEPTED
Input: "111"    -> ACCEPTED
Input: "00"      -> ACCEPTED
Input: "010"    -> ACCEPTED
Input: "0011100" -> ACCEPTED
Input: "000"    -> REJECTED
Input: "01100"          -> REJECTED
Input: "0100"  -> REJECTED
```

9. CFG: S to 0S1 mid epsilon

Language Definition: Generates strings of the form 0^n 1^n where n >= 0.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
bool is_member_g4(const char *str) {
    int len = strlen(str);
       if (len % 2 != 0) return false;
    int n = len / 2;
        for (int i = 0; i < n; i++) {
        if (str[i] != '0') return false;
    }
    for (int i = n; i < len; i++) {
        if (str[i] != '1') return false;
    }
    return true;
int main() {
    const char *test_cases[] = {"", "01", "0011", "000111", "0101", "001", "00011", "10"};
    int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);
    printf("Grammar 4: S -> 0S1 | epsilon\n");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < num_tests; i++) {
        const char *str = test_cases[i];
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g4(str) ? "ACCEPTED" : "REJECTED");
    }
    return 0;
}
```

Output:

Grammar 4: S -> 0S1 | epsilon

Input: "" -> ACCEPTED

Input: "01" -> ACCEPTED

Input: "0011" -> ACCEPTED

Input: "000111" -> ACCEPTED

Input: "0101" -> REJECTED

Input: "001" -> REJECTED

Input: "00011" -> REJECTED

Input: "10" -> REJECTED

10. CFG: S to A101A, quad A to 0A mid 1A mid epsilon

Language Definition: Generates any binary string containing "101" as a substring.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

bool is_member_g5(const char *str) {
   int len = strlen(str);
      for (int i = 0; i < len; i++) {
      if (str[i] != '0' && str[i] != '1') return false;
   }

   return strstr(str, "101") != NULL;
}

int main() {
   const char *test_cases[] = {"101", "01010", "1110100", "00101", "100", "010", "111", ""};
   int num_tests = sizeof(test_cases) / sizeof(test_cases[0]);

   printf("Grammar 5: S -> A101A, A -> 0A | 1A | epsilon\n");
   printf("-------------------------------------------------\n");
   for (int i = 0; i < num_tests; i++) {
      const char *str = test_cases[i];
```

```
        printf("Input: \"%s\" \t-> %s\n", str, is_member_g5(str) ? "ACCEPTED" : "REJECTED");
    }
    return 0;
}
```

Output:

Grammar 5: S -> A101A, A -> 0A | 1A | epsilon

Input: "101" -> ACCEPTED

Input: "01010" -> ACCEPTED

Input: "1110100" -> ACCEPTED

Input: "00101" -> ACCEPTED

Input: "100" -> REJECTED

Input: "010" -> REJECTED

Input: "111" -> REJECTED

Input: "" -> REJECTED

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application

SIMATS ENGINEERING
ASSESSMENT TOOLS

Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING
ASSESSMENT TOOLS